SpendWise — Frontend Specification

Detailed design, routes, forms, calculations, components and acceptance criteria

What this document controls

Use this after the master prompt. It asks Antigravity to build the complete frontend first, with clearly isolated mock data and no AWS backend changes.

Do not move to the backend stage until the frontend builds and the key interactions work.

COPY THE FULL PROMPT BELOW INTO ANTIGRAVITY

Use this prompt as one message. Let Antigravity inspect the repository, make the requested changes, run checks, and report what it actually completed. Do not skip the checkpoints.

```
ROLE
You are a senior product designer and Next.js engineer. Build the frontend for SpendWise — Student Expense Tracker. Inspect t

STACK
Next.js App Router, TypeScript, Tailwind CSS, Lucide React, reusable React components. Use the existing package manager. Do n

DESIGN SYSTEM
- Premium student finance product; clean, trustworthy, modern.
- Background: warm off-white; text: deep navy; accent: indigo/violet; borders: soft neutral gray.
- Use consistent spacing, card radius, typography, focus rings and button states.
- Use charts with accessible labels and sensible empty states.
- Responsive from small mobile screens through desktop.
- Avoid excessive gradients, oversized hero sections, unnecessary animation and generic dashboard templates.
- Use Indian rupee formatting by default, e.g. ■1,250.00, using Intl.NumberFormat where appropriate.

ROUTES
1. `/` — landing page, product value proposition, feature summary and clear sign-in/get-started CTA.
2. `/dashboard` — summary cards, budget progress, recent transactions, category spending chart and monthly comparison.
3. `/transactions` — list/table, search, date filter, category filter, income/expense filter, sorting and pagination or sensi
4. `/transactions/new` — create transaction form.
5. `/transactions/[id]/edit` — edit an existing transaction.
6. `/budgets` — monthly overall budget and category budget management.
7. `/analytics` — category breakdown, spending trend and current/previous-month comparison.
8. `/settings` — currency preference and profile/settings placeholder only if fully implemented.
9. Authentication routes or screens appropriate to Cognito integration; do not fake authentication.

TRANSACTION FORM
Fields: description, amount, type (INCOME or EXPENSE), category, transaction date, payment method and optional notes. Categor

BUDGETS
- Overall budget per calendar month.
- Optional category budgets per month.
- Show amount spent, limit, remaining amount and percent used.
- Warning thresholds: 75%, 90%, 100%.
- Avoid division-by-zero and negative-progress display bugs.
- Define whether the “remaining budget” means overall budget limit minus expenses, not income minus expenses, and use that de

CAN I AFFORD THIS?
Add a small calculator or card where a user enters a proposed purchase amount. Use current-month budget remaining minus the p

DATA AND COMPONENTS
- Create types for Transaction, Budget, AnalyticsSummary and API errors.
- Put temporary mock records in one dedicated module, not scattered through pages.
- Put all backend calls behind `lib/api.ts`.
- Do not claim mock data is saved remotely.
- Calculate cards and charts from the same source records so numbers remain consistent.
- Build reusable AppShell/sidebar, mobile navigation, Header, SummaryCard, TransactionList, CategoryChart, BudgetProgress, Fo
- Make controls functional; avoid decorative buttons that do nothing.
- Add semantic HTML, labels, keyboard navigation and visible focus indicators.

CHECKS
Run the project, inspect pages in a browser if available, run lint/type checks/build. Fix errors. Report files changed, local
```